Protocol Audit Report

Prepared by: Ebukar

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - [H-1] On-Chain Password Storage Exposes Sensitive Data
    - [H-2] No Access Control for `PasswordStore::setPassword` Function
  - Informational
    - [I-1] NatSpec Mismatch in `PasswordStore::getPassword`

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's password. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The Ebukar team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document corresponds with the following commit hash:

```
7d55682ddc4301a7b13ae9413095feffd9924566
```

Scope

```
./src/
└── PasswordStore.sol
```

- Solc Version: 0.8.18
- Chain(s) to deploy contract to: Ethereum

Roles

- Owner: The user who can set the password and read the password.
- Others: No other user should be able to set or read the password

Executive Summary

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] On-Chain Password Storage Exposes Sensitive Data

Description: All data stored onchain is public and visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function which is intended to be only called by the owner of the contract.

we show one such method of reading any data off chain below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: (Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool

We use `1` because that's the storage slot of `s_password` in the contract.

```
cast storage <ADDRESS_HERE> 1 --rpc-url https://127.0.0.1:8545
```

You will get an output like this:

`0x6d7950617373776f72640000000000000000000000000000000000000000000014`

You can now parse that hex to a string with:

```
cast parse-bytes32-string
0x6d7950617373776f72640000000000000000000000000000000000000000000014
```

And get an output of:

```
myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

[H-2] No Access Control for `PasswordStore::setPassword` Function

Description: The `PasswordStore::setPassword` function is set to be an `external` function. however, the natspec of the function and the overall purpose of the contract is that `this function should allow only the owner to set a new password.`

```
    function setPassword(string memory newPassword) external {
@>      // @audit There are no access control
        s_password = newPassword;
        emit SetNetPassword();
    }
```

Impact: Anyone can call the `PasswordStore::setPassword` function and set/change the password of the contract severly breaking the contract's functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test suite

▶ Code

```
    function test_anyone_can_set_password(address randomAddress) external {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory expectedPassword = "newPassword";
        passwordStore.setPassword(expectedPassword);

        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
        assertEq(actualPassword, expectedPassword);
    }
```

Recommended Mitigation: Add an access control conditional to the `setPassword` function

```
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
```

Informational

[I-1] NatSpec Mismatch in `PasswordStore::getPassword`

Description:
The `PasswordStore::getPassword` function's NatSpec documentation incorrectly references a parameter that does not exist in the function signature.

```
    /*
    * @notice This allows only the owner to retrieve the password.
@>  * @param newPassword The new password to set. // <⚠ Issue: Parameter does
```

```
not exist
    */
function getPassword() external view returns (string memory) { ... }
```

- **Function Signature**: `getPassword()`
- **Documented Parameter**: `newPassword` (does not exist).

Impact:
Developers referencing the NatSpec may expect a non-existent parameter (`newPassword`), leading to confusion during integration or debugging.

Recommended Mitigation:
Remove the `@param` line referencing the non-existent `newPassword` parameter:

```
/*
 * @notice This allows only the owner to retrieve the password.
- * @param newPassword The new password to set.
 */
```